

Parcial 2 - 2025 - Turno 2

Parcial 2 - Arquitectura de computadores 2025

Ejercicio 1

Analizar el rendimiento de un sistema que utiliza un procesador de 8 bits con un bus de direcciones de 20 bits y memoria direccionable de a byte. Se están comparando dos configuraciones de caché (Caché 1 y Caché 2), ambas con una capacidad total de almacenamiento de datos de 256 bytes y un tamaño de bloque de 32 bytes.

Caché 1: Es de Correspondencia Directa.

Caché 2: Es Asociativa por Conjuntos de 2 Vías (2-Way) y utiliza una política de sustitución LRU (Least Recently Used).

Ambas cachés están inicialmente vacías (todas las líneas marcadas como inválidas). Se ejecuta la siguiente secuencia de accesos a memoria (direcciones en hexadecimal): 0x123C2, 0x456C0, 0x123C8, 0x0456D1, 0x789C0, 0x123C4

Se Pide:

a) Para ambas configuraciones (Caché 1 y Caché 2), determine cuántos bits hay en cada campo del formato de la dirección:

	Tag	Index/Set	Word	OffsetByte
CACHE 1	12	3	5	0
CACHE 2	13	2	5	0

b) Simule la secuencia de accesos...

$$W_{mem} = 8 \text{ bits} = W_p$$

$$\text{Direcciones} = 20 \text{ bits} \Rightarrow 2^{20} \text{ direcciones}$$

$$\text{Datos} = 256 \text{ bytes} = 2^8 \cdot 2^3 \text{ bits}$$

$$\text{Bloque} = 32 \text{ bytes} \Rightarrow 32 W_p \Rightarrow 5 \text{ bits de Word (ambas)}$$

CACHE 1: (mapa directo)

$$\# \text{ Líneas} = \frac{\overset{(\text{datos})}{256 \text{ B}}}{\underset{(\text{bloque})}{32 \text{ B}}} = \frac{2^8 \cdot \cancel{2^3}}{2^5 \cdot \cancel{2^3}} = 2^3 \text{ líneas (3 bits)}$$

$$\# \text{ Tags} = \frac{\# \text{ bloques Mem}}{\# \text{ líneas}} = \frac{\frac{\# W_{mem}}{\# W_{mem} \times \text{bloque de caché}}}{\# \text{ líneas}} = \frac{2^{20}}{2^5 \cdot 2^3} = 2^{12} \text{ (12 bits)}$$

CACHE 2:

$$\# \text{ Líneas} = \frac{2^3 \text{ líneas}}{2 \text{ (vías)}} = 2^2 \text{ líneas (2 bits)}$$

$$\# \text{ tags} = \frac{2^{20}}{2^2} = 2^{18} \text{ (18 bits)}$$

b) Simule la secuencia de accesos SOLO en CACHE 2. Para cada acceso, indique si fue Hit o Miss; el número de línea o conjunto involucrado; y su estado final (contenidos del Tag y Datos) después del acceso. Para el campo de Datos utilice el formato de dirección de la primera y de la última palabra almacenada en la línea (Ej. M[0x0108] .. M[0x0100])

Acc	Address	Set	H/M	Tag (hex) V0	Datos V0	Tag (hex) V1	Datos V1
1	0x123C2	0x2	M	0x247	0x123C0 0x123DF		M[M[
2	0x456C0	0x2	M		M[M[	0x8AD	0x456C0 0x456DF
3	0x123C8	0x2	H	0x247	0x123C0 0x123DF		M[M[
4	0x0456D1	0x2	H		M[M[	0x8AD	0x456C0 0x456DF
5	0x789C0	0x2	M	0xF13	0x789C0 0x789DF		M[M[
6	0x123C4	0x2	M		M[M[	0x247	0x123C0 0x123DF

0001 0010 0011 1100 0010
 0100 0101 0110 1100 0000
 1100 1000
 0111 0000 1001 1100 0000
 1100 0100

CACHE 1

ACC. 1 : 0001 0010 0011 1100 0010

Línea: 6
 word: 2
 0x123C0 -
 0x123DF

reemplaza a

ACC. 2 : 0100 0101 0110 1100 0000

Línea 6
 word: 0
 0x456C0 -
 0x456DF

ACC. 3 : Dentro del 1° bloque almacenado **Miss** reemplaza a

ACC. 4 : Dentro del 2° bloque **Miss** reemplaza a

ACC. 5 : direc. nueva 1100 0000, Línea 6 **Miss** reemplaza a

ACC. 6 : idem ACC. 1 y 3 **Miss**

CACHE 2 : error $\frac{4}{6} = 66,67\%$

Ejercicio 2

Un compilador (aka "estudiante") debe optimizar un bucle crítico para un procesador LEGv8 con arquitectura static 2-issue. El hardware tiene un pipeline de 5 etapas (IF, ID, EX, MEM, WB) y ejecuta un paquete de dos instrucciones por ciclo, con las siguientes restricciones:

- Slot 1: Ejecuta operaciones ALU (ADD, SUB, ADDI, ORR) o Saltos (B, CBNZ).
- Slot 2: Ejecuta operaciones de Carga (LDUR) o Almacenamiento (STUR).

El procesador es estático, tiene todos los caminos de forwarding implementados, y NO genera stalls por hazards de datos. El compilador (es decir, usted) es responsable de insertar NOPs o reordenar el código para evitar conflictos.

Considere el código del Ejercicio 4. Aplicando solo la técnica de Reordenamiento de Código conforme los issue-packets a fin de MINIMIZAR los ciclos de clock necesarios para ejecutar una iteración del lazo.

Issue	Slot 1	Slot 2
1	SUBI X5, X5, #8	LDUR X9, [X5, #8]
2	SUBI X6, X6, #8	LDUR X1, [X6, #8]
3	NOP	NOP
4	ADD X2, X0, X1	NOP
5	LSR X2, X2, #1	NOP
6	NOP	STUR X2, [X5, #0]
7	CBZ X5, L1	STUR X2, [X6, #0]
8		
9		

```

1> L1: SUBI X5, X5, #8
2> SUBI X6, X6, #8
3> LDUR X0, [X5, #0]
4> LDUR X1, [X6, #0]
5> ADD X2, X0, X1
6> LSR X2, X2, #1
7> STUR X2, [X5, #0]
8> STUR X2, [X6, #0]
9> CBZ X5, L1
  
```

Ejercicio 3

Considerar que un procesador que cuenta con un predictor de dos niveles, con un GR de 4 bits y tomándose los 4 bits menos significativos del PC, que ejecuta el siguiente código en C (compilado como se muestra a la derecha):

```
do {
    // x0 <- i
    if (i % 2 == 0) { // Si i es par, no salto
        // ...
    }
    if (i % 2 != 0) { // Si i es impar, no salto
        // ...
    }
    i++;
} while (i < 3)
```

```
0x10  LOOP_FOR: ...
0x24  B.NE  FIN_IF_1
0x30  FIN_IF_1: ...
0x38  B.NE  FIN_IF_2
...
0x44  FIN_IF_2: ...
0x50  CBZ  LOOP_FOR
```

Completar la siguiente tabla con todas las posiciones de la PHT que se modifican, considerando que todas las posiciones de la tabla están inicializadas en "10" y el GR=0x07

Dirección	Contenido
0111 0100	01
1110 1000	11
1101 0000	11
1011 0100	11
0111 1000	01
1110 0000	11
1101 0100	01
1010 1000	11
0101 0000	01

Ejercicio 4

Ejercicio 4

Considerando un microprocesador out-of-order execution implementado mediante el algoritmo de Tomasulo que cuenta con las características del hardware descritas en la tabla y que ejecuta el siguiente código en assembler, muestre el contenido de las tablas de estado de las Reservation stations y registros (en la próxima página) para el 3er clock de ejecución (inclusive y considerando que se hace fetch en el clk 0). Considerar que los loads y los store se almacenan en los mismos RS (3 RS en total para los loads y store).

```
1> L1: SUBI X5, X5, #8
2>     SUBI X6, X6, #8
3>     LDUR X0, [X5, #0]
4>     LDUR X1, [X6, #0]
5>     ADD X2, X0, X1
6>     LSR X2, X2, #1
7>     STUR X2, [X5, #0]
8>     STUR X2, [X6, #0]
9>     CBZ X5, L1
```

Hardware
El salto se predice Taken correctamente
Issue = 4 instrucciones
Load y Store = 3 RS / 1 clk
ALU entera = 4 RS / 1 clk
ALU punto flotante = 4 RS / 1 clk
Multiplicación entera = 2 RS / 2 clk

(Resuelto aparte en un .xlsx)